1



Data Representation

Part I

Introduction

Computer definition:

A computer is a device that receives **input**, **processes** that input, and then produces the **output**.



IMPORTANT

For a computer to process, an **input** should be supplied first.

Input can be in form of **data** or **instructions**.

As humans, we process **analogue** data.

Computers can't process analogue data; they can only process **digital** data (0s and 1s).

KEY TERMS

Analogue Data – Data that is continuous and processed by humans. For example, sound waves.

Digital Data – Discrete data that is made up of only two values, 0 and 1.

Hardware – Physical components that make up a computer system.

Software – Programs used to operate a computer/hardware.

Examples of data:

- Number
- Text
- Image
- Audio/Sound
- Video

1. Number Systems

1.1 Denary

Definition/Features/Characteristics:

- It has a base of 10
- It uses ten values, which are 0 to 9
- Units increase by the power of 10

1.2 Binary

Definition/Features/Characteristics:

- It has a base of 2
- It only uses two values, which are 0 and 1
- Units increase by the power of 2

IMPORTANT

Any form of data needs to be converted to binary to be understood and processed by the computer.

Why is data stored as binary in computers?

- As computers consist of logic gates
- Logic gates can only **process** data in **two states** (0 and 1)

1.2.1 Conversion from Denary to Binary

Example:

We need to represent the denary number **25** in computer memory.

This means that we need to convert the number 25 to binary.

Solution:

This is done using the table method:

1 Powers of 2 (starting with 2^0) are placed as column headings

512	256	128	64	32	16	8	4	2	1

2 1s are placed in the appropriate columns so that if column headings are added, the result is 25.

512	256	128	64	32	16	8	4	2	1
					1	**1**	**0**	**0**	**1**

16 + 8 + 1 = 25

This means that the denary value 25 is converted to 11001 in binary

Another example:

Convert **125** to binary.

Solution:

Scan the following QR code for a more elaborate answer:

1.2.2 Conversion from Binary to Denary

Example:

We need to convert the binary number **100110** to denary.

Solution:

1 Powers of 2 (starting with 2^0) are placed as column headings

512	256	128	64	32	16	8	4	2	1

2 Correctly place a 1 or a 0 for each column

512	256	128	64	32	16	8	4	2	1
				1	**0**	**0**	**1**	**1**	**0**

3 Add the denary values for columns containing 1s

32 + 4 + 2 = 38

4 This will give a total which is the answer **38**

1

TEAM

Data Representation

Part II

1.3 Hexadecimal

Definition/Features/Characteristics:

- It has a base of 16
- It uses sixteen values, which are 0 to 9 and A to F
- Units increase by the power of 16

Hexadecimal	Denary
A	10
B	11
C	12
D	13
E	14
F	15

1.3.1 Conversion from Hexadecimal to Binary

Example:

Convert the hexadecimal number **9A** to binary.

Solution:

Each hexadecimal digit is represented in 4 binary digits.

This means that the number 9A is represented in 8 binary digits.



Table method is then used to convert each hexadecimal digit to its equivalent 4 binary digits:

8	4	2	1
1	0	0	1

9

8	4	2	1
1	0	1	0

A

The answer is: 1001 1010

Another example:

Convert **F0C** to binary.

Solution:

Scan the following QR code for a more elaborate answer:

1.3.2 Conversion from Binary to Hexadecimal

Example:

Convert the binary number 100110110 to hexadecimal.

Solution:

We recall that **one hexadecimal digit is represented in 4 binary digits**.

For this problem, we need to **group** every 4 digits and represent them as one hexadecimal digit.

0001 0011 0110

After grouping each of 4 binary digits, we know that we have **3 hexadecimal digits**.

We use the table method to figure them out:

8	4	2	1
0	0	0	1

1

8	4	2	1
0	0	1	1

3

8	4	2	1
0	1	1	0

6

The answer is 136

1.3.3 Conversion from Hexadecimal to Denary

We convert from hexadecimal to binary first, then from binary to denary.

Hexadecimal

↓

Binary

↓

Denary

Example:

Convert the hexadecimal number **FB** to denary.

Solution:

1 Convert **FB** to binary:

8	4	2	1
1	**1**	**1**	**1**

F

8	4	2	1
1	**0**	**1**	**1**

B

FB = 1111 1011

2 Convert the binary number 11111011 to denary:

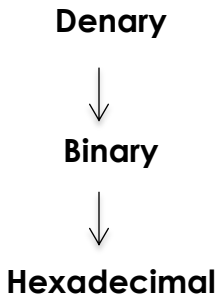
512	256	128	64	32	16	8	4	2	1
		1	**1**	**1**	**1**	**1**	**0**	**1**	**1**

128 + 64 + 32 + 16 + 8 + 2 + 1 = 251

3 The answer is **251**

1.3.4 Conversion from Denary to Hexadecimal

We convert from denary to binary first, then from binary to hexadecimal.



Example:

Convert the denary number **150** to hexadecimal.

Solution:

1 Convert **150** to binary:

512	256	128	64	32	16	8	4	2	1
		1	**0**	**0**	**1**	**0**	**1**	**1**	**0**

128 + 16 + 4 + 2 = 150

2 Convert 1001 0110 to hexadecimal

8	4	2	1
1	**0**	**0**	**1**

9

8	4	2	1
0	**1**	**1**	**0**

6

3 The result is **96** in hexadecimal

Why is hexadecimal preferred by programmers rather than binary?

As hexadecimal is a shorter representation of binary, so:

- It is easier for programmers to read/understand/write
- It is easier to debug (locate errors)
- It takes up less space **on the screen/on paper**

Uses of hexadecimal:

- HTML **colour codes**
- Error messages
- MAC addresses
- IPv6 addresses
- ASCII // Unicode
- Locations in memory
- Debugging

1



Data Representation

Part III

Memory Units

Bit

Definition:

Refers to **one binary digit**.

Example:

For example, the following binary number is composed of 5 bits:

11010

Nibble

Definition:

A group of **4 bits**.

Byte

Definition:

A group of **8 bits**.

The following table shows more memory units:

Name of memory unit	Number of bytes	Number of bytes
1 Kibibyte (KiB)	2^{10}	1024 bytes
1 Mebibyte (MiB)	2^{20}	1024 KiB
1 Gibibyte (GiB)	2^{30}	1024 MiB
1 Tebibyte (TiB)	2^{40}	1024 GiB
1 Pebibyte (PiB)	2^{50}	1024 TiB
1 Exbibyte (EiB)	2^{60}	1024 PiB

2. Text

Text is represented using one of the following **character sets**:

- ASCII (American Standard Code for Information Exchange)
- Unicode

Character set definition:

- A **list** of all characters and symbols that can be represented by a computer system
- **Each character**/symbol is assigned a **unique value**.

2.1 ASCII

Definition:

- A character set for all the characters on a standard English keyboard
- Each character is represented using **8 bits (1 byte)**

The following is the **ASCII table** in which each character is represented with a unique value:

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
32	20	<SPACE>	64	40	@	96	60	`
33	21	!	65	41	A	97	61	a
34	22	"	66	42	B	98	62	b
35	23	#	67	43	C	99	63	c
36	24	$	68	44	D	100	64	d
37	25	%	69	45	E	101	65	e
38	26	&	70	46	F	102	66	f
39	27	'	71	47	G	103	67	g
40	28	(	72	48	H	104	68	h
41	29	)	73	49	I	105	69	i
42	2A	*	74	4A	J	106	6A	j
43	2B	+	75	4B	K	107	6B	k
44	2C	,	76	4C	L	108	6C	l
45	2D	-	77	4D	M	109	6D	m
46	2E	.	78	4E	N	110	6E	n
47	2F	/	79	4F	O	111	6F	o
48	30	0	80	50	P	112	70	p
49	31	1	81	51	Q	113	71	q
50	32	2	82	52	R	114	72	r
51	33	3	83	53	S	115	73	s
52	34	4	84	54	T	116	74	t
53	35	5	85	55	U	117	75	u
54	36	6	86	56	V	118	76	v
55	37	7	87	57	W	119	77	w
56	38	8	88	58	X	120	78	x
57	39	9	89	59	Y	121	79	y
58	3A	:	90	5A	Z	122	7A	z
59	3B	;	91	5B	[	123	7B	{
60	3C	<	92	5C	\	124	7C	|
61	3D	=	93	5D	]	125	7D	}
62	3E	>	94	5E	^	126	7E	~
63	3F	?	95	5F	_	127	7F	<DELETE>

Example:

Represent the word **Computer** in memory.

Solution:

After checking the ASCII table, the word **Computer** is represented as follows:

In denary:

67 111 109 112 117 116 101 114

In hexadecimal:

43 6F 6D 70 75 74 65 72

In binary:

01000011 01101111 01101101 01110000 01110101 01110100 01100101 01110010

2.2 Unicode

Definition:

- A character set which can represent all the languages of the world
- Each character is represented using up to 32 bits (4 bytes)

Differences between ASCII character set and Unicode character set:

- Unicode has more characters than ASCII
- Unicode includes more languages than ASCII
- Unicode includes emojis which are not included in ASCII
- Unicode requires **more bits per character** than ASCII

Why does text stored using Unicode require more storage space than ASCII:

Unicode requires **more bits per character** than ASCII.

A sample of Unicode characters are shown in the following table:

U+00A0 NBSP 160	U+00A1 ¡ 161	U+00A2 ¢ 162	U+00A3 £ 163	U+00A4 ¤ 164	U+00A5 ¥ 165	U+00A6 ¦ 166	U+00A7 § 167	U+00A8 ¨ 168	U+00A9 © 169	U+00AA ª 170	U+00AB « 171	U+00AC ¬ 172	U+00AD 173	U+00AE ® 174	U+00AF ¯ 175
U+00B0 ° 176	U+00B1 ± 177	U+00B2 ² 178	U+00B3 ³ 179	U+00B4 ´ 180	U+00B5 µ 181	U+00B6 ¶ 182	U+00B7 · 183	U+00B8 ¸ 184	U+00B9 ¹ 185	U+00BA º 186	U+00BB » 187	U+00BC ¼ 188	U+00BD ½ 189	U+00BE ¾ 190	U+00BF ¿ 191
U+00C0 À 192	U+00C1 Á 193	U+00C2 Â 194	U+00C3 Ã 195	U+00C4 Ä 196	U+00C5 Å 197	U+00C6 Æ 198	U+00C7 Ç 199	U+00C8 È 200	U+00C9 É 201	U+00CA Ê 202	U+00CB Ë 203	U+00CC Ì 204	U+00CD Í 205	U+00CE Î 206	U+00CF Ï 207
U+00D0 Ð 208	U+00D1 Ñ 209	U+00D2 Ò 210	U+00D3 Ó 211	U+00D4 Ô 212	U+00D5 Õ 213	U+00D6 Ö 214	U+00D7 × 215	U+00D8 Ø 216	U+00D9 Ù 217	U+00DA Ú 218	U+00DB Û 219	U+00DC Ü 220	U+00DD Ý 221	U+00DE Þ 222	U+00DF ß 223
U+00E0 à 224	U+00E1 á 225	U+00E2 â 226	U+00E3 ã 227	U+00E4 ä 228	U+00E5 å 229	U+00E6 æ 230	U+00E7 ç 231	U+00E8 è 232	U+00E9 é 233	U+00EA ê 234	U+00EB ë 235	U+00EC ì 236	U+00ED í 237	U+00EE î 238	U+00EF ï 239
U+00F0 ð 240	U+00F1 ñ 241	U+00F2 ò 242	U+00F3 ó 243	U+00F4 ô 244	U+00F5 õ 245	U+00F6 ö 246	U+00F7 ÷ 247	U+00F8 ø 248	U+00F9 ù 249	U+00FA ú 250	U+00FB û 251	U+00FC ü 252	U+00FD ý 253	U+00FE þ 254	U+00FF ÿ 255

How text is converted to binary:

- A character set is used
- ⋯ such as Unicode/ASCII
- Each character has a **unique binary** value
- Binary values of characters are stored in **sequence** in the file

3. Images

Definition of an image:

An image is a series of **pixels**.

Important note:

The file size and quality of the image depend on the **image resolution** and the **colour depth.**

KEY TERMS

Pixel – The smallest component of an image.

Image Resolution – The dimensions of the image.

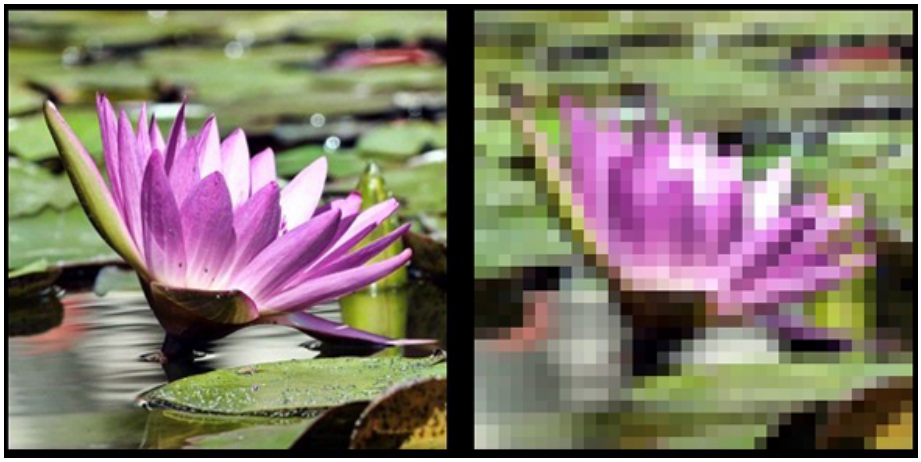
Colour Depth – The number of bits used to represent **each colour** in the image.

Metadata – Extra information saved with the main data that describes its properties.

Examples of Metadata:

- File Name
- File Type
- File Size
- Date Created

High-resolution image vs. low-resolution image



Example of different colour depths



How an image is converted to binary:

- Each pixel represents a colour and is **given a binary value**
- The image resolution is set..
- .. that is the dimensions of the image
- The colour depth is set..
- ..that is the number of bits used to represent each colour
- Pixels are stored in sequence in the file

The effect of increasing the image resolution on an image:

- More pixels in the image, so ..
  - .. the image will have more detail
  - .. the image will be closer to real-life
  - .. the file size will increase

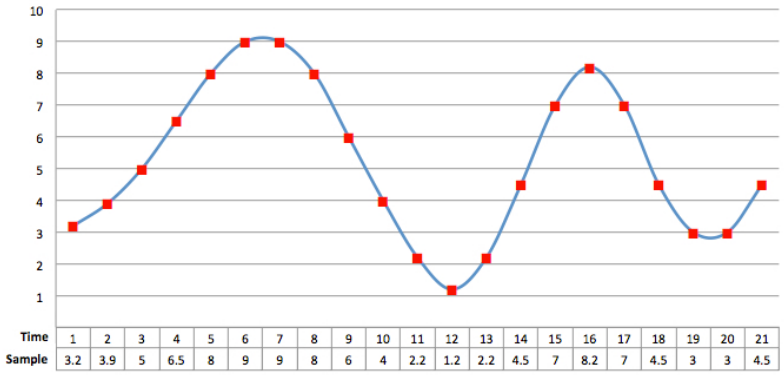
The effect of increasing the colour depth on an image:

- More bits are allocated to **each pixel**, so..
  - .. a greater range of colours can be seen in the image
  - .. the image will be closer to real-life
  - .. the file size will increase

4. Audio

How audio is converted to binary:

- The **amplitude** of the sound wave is measured at set time intervals
- Each amplitude has a unique binary value
- The **sampling rate** is set
- ⋯ that is the number of samples taken per second
- The **sampling resolution** is set
- ⋯ that is the number of bits used to represent each sample
- Each sample taken is converted to binary



Time	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
Sample	3.2	3.9	5	6.5	8	9	9	8	6	4	2.2	1.2	2.2	4.5	7	8.2	7	4.5	3	3	4.5

KEY TERMS

Sampling – The measurement of the amplitude of a sound wave at regular time intervals.

Sampling Rate – The number of samples taken per second.

Definition of sampling resolution:

- The number of bits used to represent **each sample**
- .. that defines the number of different **amplitudes** that can be recorded

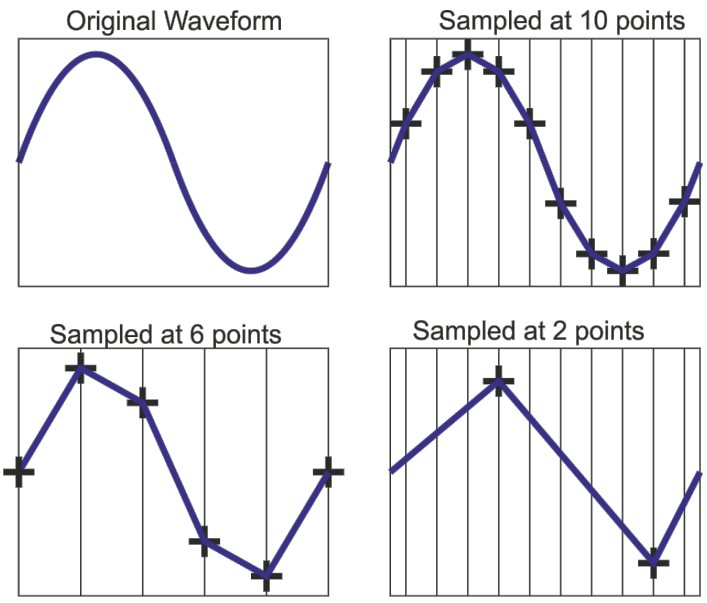
The effect of increasing the sampling resolution on the audio:

- More bits are allocated to **each amplitude** so..
  - .. more accurate **amplitudes**
  - .. a wider range of **amplitudes** can be recorded
  - .. the file size will increase

The effect of increasing the sampling rate on the audio:

- More **amplitudes** are taken **per second,** so..
  - .. more accurate sound // closer to the original sound
  - .. the file size will increase

Example of different sampling rates



1



Data Representation

Part IV

5. Binary Operations

5.1 Register

Definition:

A component in the CPU that is used to **store** data **temporarily**.

Example:

In this example, a 4-bit register is used to store the binary value **1000**.

Bit 4	Bit 3	Bit 2	Bit 1
1	0	0	0

- Bit 1 is called the **least significant bit,** as it has the least effect on the value of the register
- Bit 4 is called the **most significant bit,** as it has the highest effect on the value of the register

KEY TERMS

Most Significant Bit – The leftmost bit in a binary value.
Least Significant Bit – The rightmost bit in a binary value.

5.2 Binary Addition

To add **two** binary digits:

Addition	Sum	Carry
0 + 0	0	0
0 + 1	1	0
1 + 0	1	0
1 + 1	0	1

To add **three** binary digits:

Addition	Sum	Carry
0 + 0 + 0	0	0
0 + 0 + 1	1	0
0 + 1 + 0	1	0
0 + 1 + 1	0	1
1 + 0 + 0	1	0
1 + 0 + 1	0	1
1 + 1 + 0	0	1
1 + 1 + 1	1	1

Example 1:

The addition of two 8-bit binary numbers can be done as follows:

Carry	0	0	0	0	1	1	1	0	0
Number 1		**0**	**0**	**1**	**0**	**0**	**1**	**1**	**1**
Number 2		**0**	**1**	**0**	**0**	**1**	**0**	**1**	**0**
Result		0	1	1	1	0	0	0	1

+

0010 0111 + 0100 1010 = 0111 0001

Example 2:

The addition of two 8-bit binary numbers can be done as follows:

Carry	1	1	1	1	1	1	1	0	0
Number 1		0	1	1	0	1	1	1	0
Number 2		1	1	0	1	1	1	1	0
Result	1	0	1	0	0	1	1	0	0

+

Note:

The addition of **0110 1110 & 1101 1110** resulted in a number whose value is greater than 255. A number greater than 255 cannot be stored in an 8-bit register.

This is called an **overflow error**.

Definition of overflow error:

A type of error that occurs when a number larger than a register can store is generated.

Why did overflow occur in the previous example?

- The result of the calculation is greater than 255..
- .. which would require more than **8 bits** to be stored

5.3 Two's complement

Definition:

A numbering system used to represent both **positive and negative binary numbers**.

Example 1:

Represent the negative number **-54** in an 8-bit register

1 To convert a negative number, we first convert it as if it was positive:

128	64	32	16	8	4	2	1
0	**0**	**1**	**1**	**0**	**1**	**1**	**0**

2 Then we flip all the values (each 1 is converted into 0, and each 0 is converted to 1)

128	64	32	16	8	4	2	1
1	**1**	**0**	**0**	**1**	**0**	**0**	**1**

3 Then add 1 (in the least significant bit) to the binary number

Carry	**0**	0	0	0	0	0	1	0	
Flipped number	**1**	**1**	**0**	**0**	**1**	**0**	**0**	**1**	**+**
Add 1	0	0	0	0	0	0	0	1	
Result	1	1	0	0	1	0	1	0	

The denary number **-54** in two's complement is **11001010**

IMPORTANT

The most significant bit in two's complement represents the sign of the number.

Negative numbers have a most significant bit of **1**, while positive numbers have a most significant bit of **0**.

Example 2:

Convert the two's complement 8-bit binary number **00111011** to a denary number.

1 Place the binary number in the two's complement table

128	64	32	16	8	4	2	1
0	**0**	**1**	**1**	**1**	**0**	**1**	**1**

2 Add the denary values for columns containing 1s

32 + 16 + 8 + 2 + 1 = 59

Note that this number had a most significant bit of **0,** so the number is a positive number.

Example 3:

Convert the two's complement 8-bit binary number **10101001** to a denary number.

Steps:

1 Check the most significant bit of the binary number.

This number has a most significant bit of **1,** so the number is a negative number converting negative numbers:

2 Take the two's complement of the number, by flipping the number and adding 1

Carry	0	0	0	0	0	0	0	0	
Flipped number	**0**	**1**	**0**	**1**	**0**	**1**	**1**	**0**	**+**
Add 1	0	0	0	0	0	0	0	1	
Result	**0**	**1**	**0**	**1**	**0**	**1**	**1**	**1**	

3 Convert the result from binary to denary normally

128	64	32	16	8	4	2	1
0	**1**	**0**	**1**	**0**	**1**	**1**	**1**

64 + 16 + 4 + 2 + 1 = 87

4 Place the correct negative sign so the number is **-87**

IMPORTANT

The **minimum** denary value that can be represented using two's complement in 8 bits is **-128**

The **maximum** denary value that can be represented using two's complement in 8 bits is **127**

5.4 Logical Shift

Definition:

A binary operation that moves bits to some places to the left or right.

5.4.1 Logical Left Shift

Process of logical left shift one place:

- **Each bit** in the binary number is **shifted/moved** to the left
- The **most significant bit (left most bit)** is lost
- A **zero** is added as the **least significant bit (right most bit)**

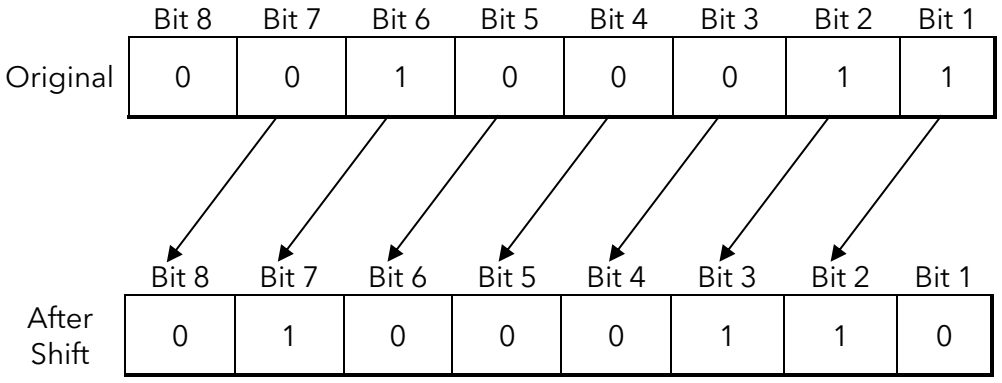
IMPORTANT

The effect of **left** logical shift is

Multiplication of the denary value by $2^{the\ number\ of\ shifted\ places}$

Example 1:

The register below is to be shifted **one place** to the left



Effect of logical left shift:

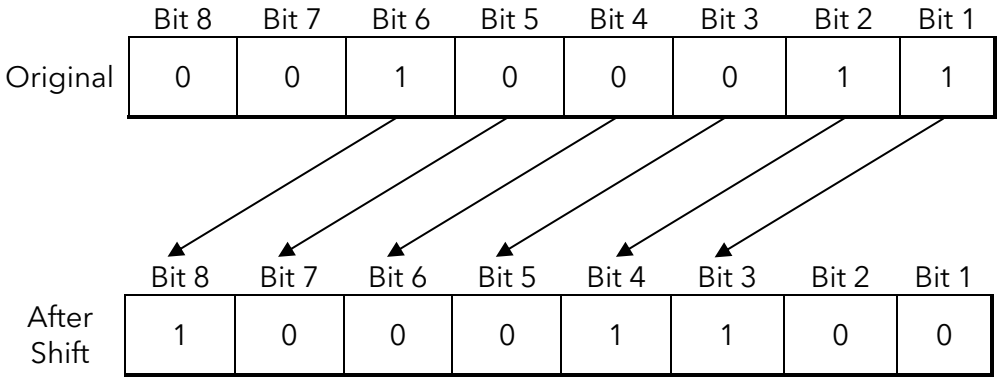
In the previous example,

- The denary value after the shift is **multiplied by 2** to be 70

A 1-bit logical left shift causes the denary number to **be multiplied by 2**

Example 2:

The register below is to be shifted two places to the left.



Effect of logical left shift in the previous example:

In the previous example,

- The denary value after the shift is **multiplied by 4** to be 140

This means that a 2-bit logical left shift causes the denary number to be **multiplied by 4** (2^2)

5.4.2 Logical Right Shift

Process of logical right shift one place:

- **Each bit** in the binary number is **shifted/moved** to the right
- The **least significant bit (right most bit)** is lost
- A **zero** is added as the **most significant bit (left most bit)**

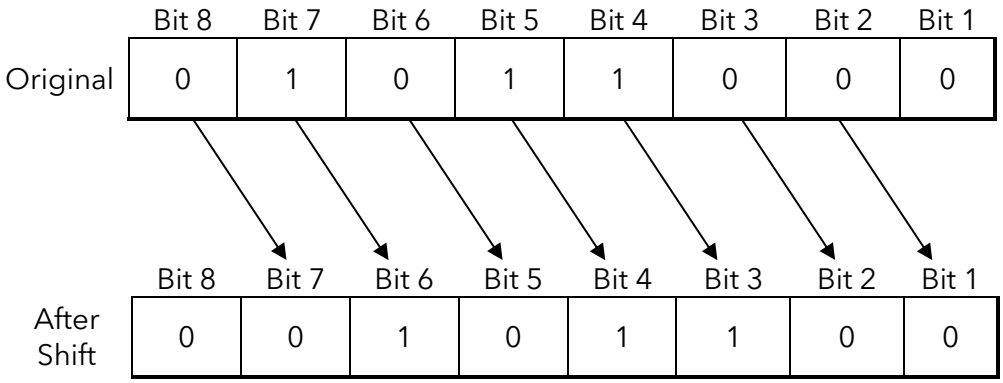
IMPORTANT

The effect of logical **right** shift is

Division of the denary value by $2^{the\ number\ of\ shifted\ places}$

Example 1:

The register below is to be shifted one place to the right.



Effect of logical right shift:

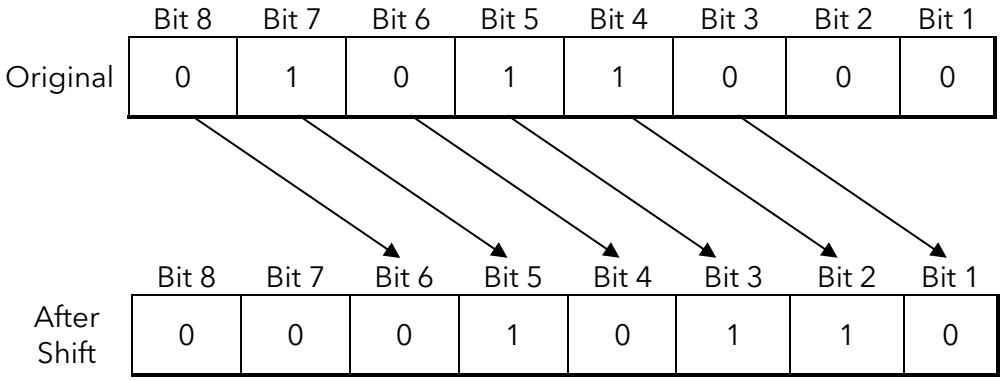
In the previous example,

- The denary value after the shift is **divided by 2** to be 44

This means that a 1-bit logical right shift causes the denary number to be **divided by 2**

Example 2:

The register below is to be shifted two places to the right.



Effect of logical right shift:

In the previous example,

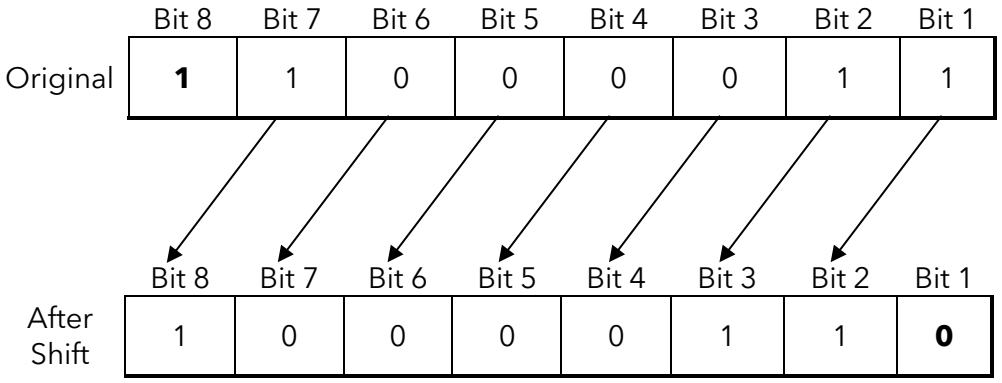
- The denary value after the shift is **divided by 4** to be 22

This means that a 2-bit logical right shift causes the denary number to be **divided by 4** (2^2)

5.4.3 Issues with Logical Shift

Example:

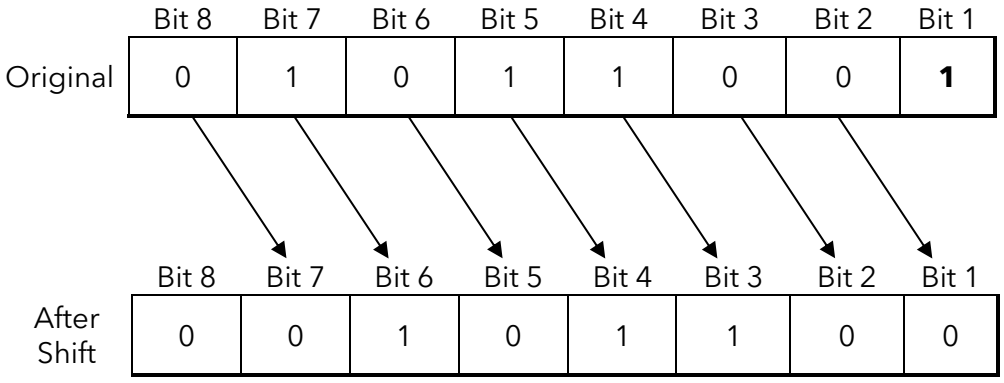
The register below is to be shifted one place to the left.



The value becomes **inaccurate** as the most significant bit (Bit 8) is lost

Example 2:

The register below is to be shifted one place to the right.

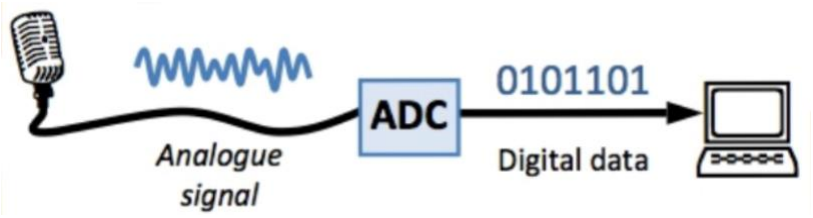


The value becomes **inaccurate** as the least significant bit (Bit 1) is lost

Analogue and Digital data

Analogue to Digital Converter (ADC)

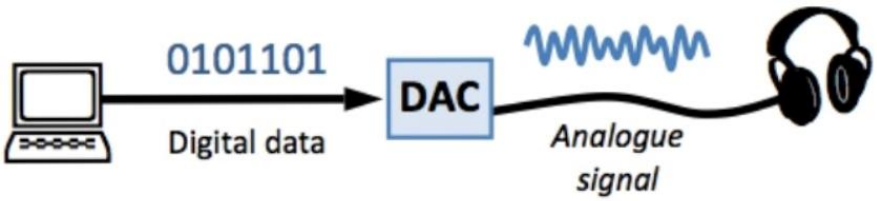
Microphone



ADC is needed to convert the microphone's analogue signals into digital data that the computer can then **process**.

Digital to Analogue Converter (DAC)

Headphones/Speakers



DAC converts the digital file from the computer into analogue signals, which the loudspeaker then converts into sound to be **understood by humans**.

KEY TERMS
Analogue Data – Data that is continuous and processed by humans. For example, sound waves
Digital Data – Discrete data that is made up of only two values, 0 and 1.
ADC – It is a device that converts analogue data to digital data.
DAC – It is a device that converts digital data to analogue data.